

Project Report: Multitask Learning for Geometric Shape Classification and Counting

Anastasiya Khodzina

1. Introduction & Exploratory Data Analysis (EDA)

The goal of this project is to design, implement, and evaluate a Multitask Neural Network capable of simultaneously classifying pairs of geometric shapes and counting their occurrences in 28x28 binary images.

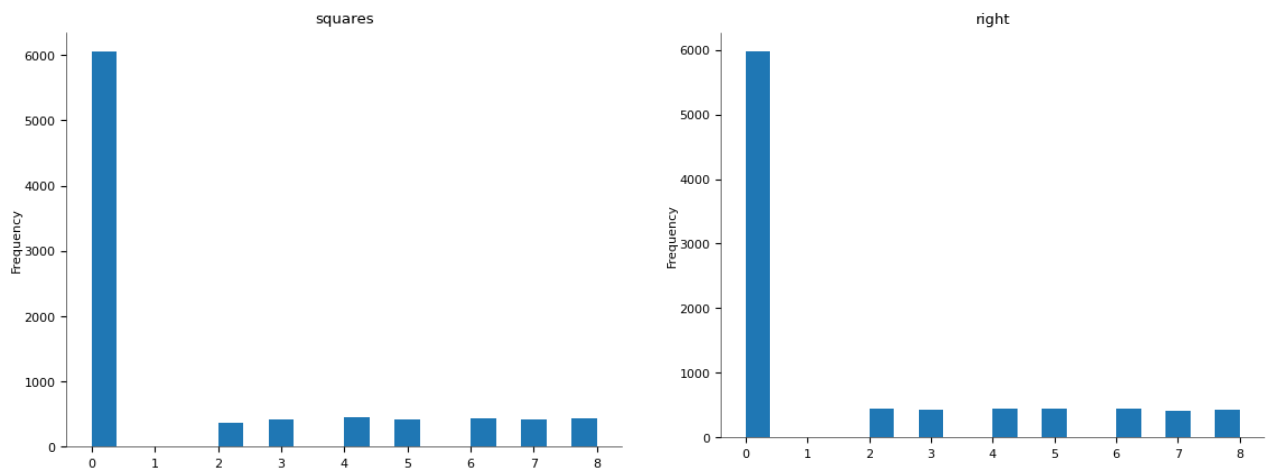
Dataset Analysis

The GSN (Geometric Shape Numbers) dataset consists of black-and-white images each containing exactly two types of geometric shapes chosen from six classes: Square, Circle, Triangle Up, Triangle Right, Triangle Down, and Triangle Left.

- **Image Dimensions:** 28x28x1 (grayscale).
- **Label Structure:** Each image has a classification label (one of 135 configurations of two shapes summing to 10 items) and a regression label (a vector of 6 counts).
- **Data Split:** 9,000 images for training and 1,000 for validation.

EDA Observations: Visual inspection of the dataset reveals that the images are very low-resolution and contain noise. The shapes often overlap, making the counting task non-trivial. The class distribution for shape pairs is combinatorial, requiring the model to learn distinguishing features for small, low-resolution geometric figures.

Some information about the data (we see that there are no images with exactly one square or right triangle):



2. Model Architecture

I designed a multitask deep learning model using PyTorch. The architecture consists of a shared backbone for feature extraction and two task-specific heads.

1. Shared Backbone:

The backbone is a Convolutional Neural Network (CNN) strictly following the assignment constraints:

- Four convolutional blocks: Conv2d (channels: 1->8->16->32->64) followed by ReLU activations.
- Flattening layer.
- A fully connected layer (Linear) mapping the flattened features to a 256-dimensional vector.

This shared representation captures spatial hierarchies and geometric features relevant to both tasks.

2. Task-Specific Heads:

- *Classification Head (head_cls)*: Receives the 256-dim embedding and outputs log-probabilities for 135 classes using LogSoftmax. Dropout was applied to prevent overfitting.
- *Regression Head (head_cnt)*: Receives the same 256-dim embedding and outputs a 6-dimensional vector representing the counts for each shape type.

3. Joint Loss Function:

The model is trained to minimize a weighted sum of losses:

$$Loss = NLLLoss(classification) + \lambda * SmoothL1Loss(regression)$$

For the multitask experiment, $\lambda = 0.5$ was chosen to balance the gradients from both tasks.

3. Data Preprocessing and Augmentation

To improve model robustness and generalization, a custom Dataset class was implemented. The following augmentations were applied during training:

1. *Random Horizontal Flip (p=0.5)*: Increases data diversity. Crucially, labels for direction-sensitive shapes ("Triangle Left" and "Triangle Right") are swapped when a flip occurs.
2. *Gaussian Noise (sigma=0.02)*: Adds mild noise to input images. This acts as a regularization technique, preventing the model from memorizing specific pixel artifacts. A low sigma was selected to avoid destroying the structural information of the small 28x28 shapes.
3. *Random Erasing (p=0.4)*: Simulates occlusion by erasing small rectangles (scale 0.001–0.01). This forces the network to recognize shapes even when partially visible.

4. Experiments and Results

Three experimental settings were conducted using the Adam optimizer (*learning rate* = $1e-3$). Early stopping was implemented to prevent overfitting and save computational resources.

Experimental Settings:

1. Multitask: $\lambda_{cnt} = 0.5$ (Both tasks active).

2. Classification-only: $\lambda_{cnt} = 0$ (Regression loss ignored).
3. Regression-only: Classification loss ignored (optimized only for RMSE).

Results Summary

Metric	Multitask Model ($\lambda=0.5$)	Classification Only	Regression Only
Top-1 Accuracy	30.5%	2.6%	1.1%
Macro F1-score	0.23	0.00	0.00
Overall RMSE	2.38	15.66	4.93
Training Status	Converged	Failed to converge	Failed to converge

Analysis of Results

Multitask Model:

This model performed significantly better than the single-task baselines. It achieved a validation accuracy of 30.5% and an RMSE of 2.38. The Per-pair accuracy metric showed that the model was highly effective at distinguishing distinct shapes (e.g., "Triangle Up + Triangle Down" had 97.01% accuracy) but struggled with certain combinations like "Triangle Right + Triangle Down" (~66%).

Single-Task Baselines:

Both single-task models failed to learn effectively compared to the multitask approach.

- Classification-only: Achieved only 2.6% accuracy with a very high RMSE (15.66), indicating the model could not extract meaningful features for either task without the regression guidance.
- Regression-only: Achieved an RMSE of 4.93, which is significantly worse than the Multitask model's result (2.38). Classification accuracy was near random (~1%), as expected since the classification head was not trained.

The results show that the learning rate of 1e-3 was too high for the single-task models. They could not learn effectively. However, adding the second (auxiliary) task stabilized the training and helped the model find a good solution.

Approximate Runtime (Updated for T4)

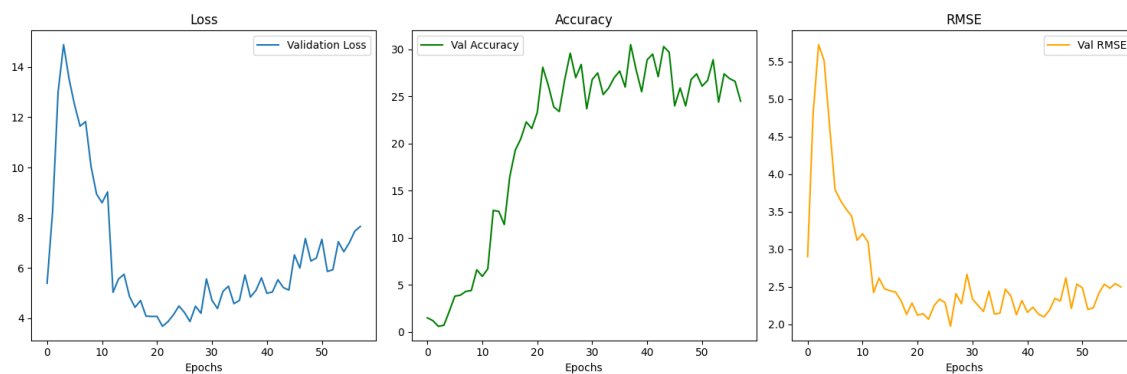
Based on my execution times on NVIDIA A100, here are the estimated runtimes for a standard NVIDIA T4 GPU (approx. 5x factor).

Experiment	Training Time (A100)	Est. Time (T4 GPU)
Multitask Model	~413 sec	~35 min
Classification-only	~156 sec	~13 min
Regression-only	~147 sec	~12 min

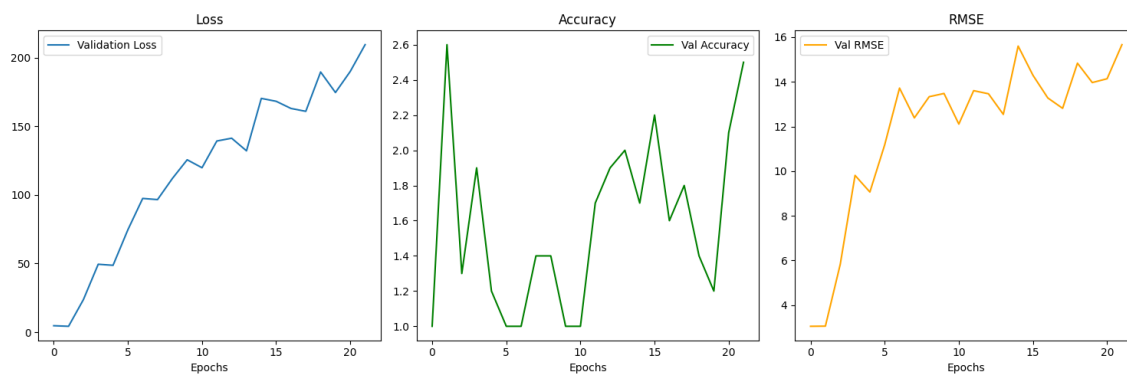
Total Execution Time: ~1 hour.

5.Learning Curves

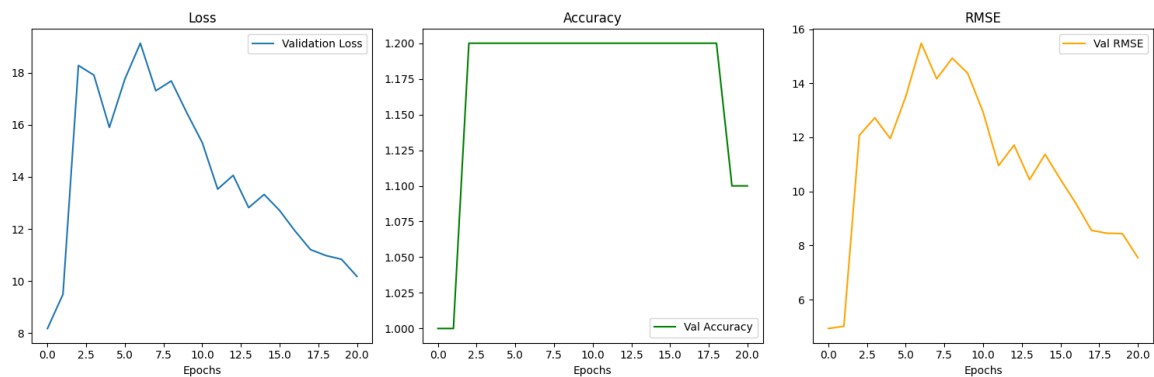
Multitask Model:



Classification Model:



Regression Model:



6. Discussion on Multitask Effects

1. *Better Learning and Stability* The most important finding is that multitask learning made the training stable. The network had to do two things at once: identify the shapes and count them. Because of this, the shared part of the model (the backbone) learned much better features. The counting task (regression) helped the classification task. This makes sense because knowing the exact number of each specific shape is essential for deciding *which class* the image belongs to.
2. *Why it Converged* The single-task models failed to learn. They likely got stuck or were too unstable because the learning rate was high. However, the joint loss (combining both tasks) gave the network a clearer signal to learn from. This allowed the multitask model to find a good solution, while the separate models learned nothing meaningful.
3. *Conclusion* This project proves that multitask learning is effective. Adding the extra "counting" task significantly improved the main "classification" task compared to the single-task baseline. Because the tasks shared the same backbone, they were able to help each other learn, resulting in a functional model.